

The enterprise AI stack - a cheatsheet

Everything you need to know to have an informed conversation about AI in any enterprise setting. No jargon for the sake of jargon. If a concept is simple, it stays simple.

The 4 levels (and one you should probably ignore)

These are not steps you climb in order. They're a menu. Each business problem picks its own level. Most companies need level 1. Some need level 4. Almost none need level 5.

#	Level	What it does	When to use it	Cost (India)	Time
1	RAG	Connects an LLM to your company's data. System finds relevant documents and feeds them to the model before it answers. No training. Pure wiring.	Employees need to search internal docs, policies, contracts or knowledge bases using natural language.	10-30L to build 2-5L/month to run	4-8 weeks
2	Fine-tuning	Changes how the model behaves - its tone, format, terminology. You feed it examples of how you want it to write. It learns your style. Baked into weights, permanent.	AI output needs to sound like your brand, follow your format or use your domain language. Generic AI tone isn't acceptable.	5-15L per cycle	2-4 weeks per cycle
3	Distillation	A big expensive model (teacher) answers thousands of questions. A small cheap model (student) learns from those answers. Result: you get 80% of the big model's quality at 10% of the cost.	You're paying too much for API calls. You need the AI to run faster, cheaper or offline. You want to stop depending on an external provider.	10-25L	4-6 weeks
4	Classical ML	Traditional machine learning on structured data. Not LLMs - algorithms like random forests, gradient boosting, logistic regression. Predicts outcomes from patterns in numbers.	You have structured data (spreadsheets, logs, transactions) and want to predict outcomes - churn, risk, demand or failure.	20-50L to build 5-10L/month	3-6 months
5	Own model	Train a base LLM from scratch. 99% of companies should ignore this entirely.	Only if your product IS the model (Anthropic, OpenAI, Sarvam). Otherwise, don't.	50-100Cr+	12-18 months

Services companies sell them level 5 ambition at 50Cr. The person who right-sizes the solution to the actual problem - that's who earns trust.

The agent layer - what sits on top

An agent is not a level. It's the orchestration layer that ties the levels together. You give it a goal in plain English. It breaks the goal into steps, picks the right level for each step, runs them in order and gives you one combined result.

Think of it like a project manager. You say "prepare me for tomorrow's meeting." The PM doesn't do the work himself. He looks at his team (RAG for search, fine-tuned model for writing, prediction engine for anticipation), assigns each task to the right person, collects the output and delivers one clean briefing.

The 5 agent concepts

These are the building blocks of any AI agent - from a laptop project to ChatGPT plugins to enterprise automation systems. The names are standard across the industry.

Concept	What it means	Industry term
Task decomposition	Breaking a vague goal into concrete steps. "Prepare for meeting" becomes: predict files, search documents, write briefing.	Same everywhere. LangChain, CrewAI and AutoGen all use this term.
State management	Tracking what's done and what's next. A simple checklist: pending, running, done or failed.	Also called agent memory or scratchpad.
Tool selection	For each step, picking the right function to call. Search for finding, write for generating, predict for anticipating.	Called tool calling or function calling. When OpenAI or Anthropic say this, same concept.
Error recovery	When a tool fails, the agent doesn't stop. It logs the failure, skips the step and continues with the rest.	Called fault tolerance or graceful degradation in production systems.
Planning loop	Each step receives results from all previous steps as context. Step 3 knows what step 1 predicted and what step 2 found.	Called ReAct (reason + act) in research papers.

The agent's quality depends on two things: how smart the planner model is (plan quality) and how good the tools are (execution quality). Better models make better plans. Better data makes better tools. The framework stays the same.

Key concepts - what each word actually means

Embeddings

A way to turn text into numbers so machines can compare meaning. The sentence "I'm hungry" and "I want food" look different as text but their embeddings (number representations) are very close together. This is how RAG finds the right documents - it compares the embedding of your question against the embeddings of every chunk in your database. Closest match wins.

Vector database

A database optimized for storing and searching embeddings. Regular databases search by exact match (find all rows where name = "Swapnil"). Vector databases search by similarity (find all chunks whose meaning is closest to this question). ChromaDB, Pinecone and Weaviate all do the same thing.

Chunking

Breaking large documents into smaller pieces before embedding them. You can't feed an entire 200-page PDF into an embedding model - it has a size limit. So you split it into paragraphs or sections (chunks), embed each chunk separately and store them individually. When someone asks a question, the system retrieves the most relevant chunks, not the whole document.

Context window

The maximum amount of text an LLM can read at once. Think of it as the model's working memory. GPT-4 can handle ~128K tokens (~100 pages). Mistral 7B handles ~8K tokens (~6 pages). If your retrieved chunks plus the question exceed the context window, the model can't read all of it. This is why chunk size matters - smaller chunks mean more fit in the window.

Tokens

The unit LLMs think in. Not words - pieces of words. "Enterprise" is 2-3 tokens. "AI" is 1 token. Roughly: 1 token is about 3/4 of a word. 1,000 tokens is about 750 words. You pay API providers per token. You hit context window limits in tokens. Everything is measured in tokens.

LoRA (low-rank adaptation)

A technique that makes fine-tuning possible on normal hardware. Instead of retraining all 7 billion parameters of a model (which needs massive GPUs), LoRA freezes 99.9% of the weights and only trains a tiny adapter layer (0.1%). The result is nearly as good as full fine-tuning but runs on a laptop. This is how Phase 2 of this project works.

Temperature

A setting that controls how creative vs predictable the model is. Temperature 0 = always picks the most likely next word (factual, repetitive). Temperature 1 = more randomness (creative, sometimes wrong). For RAG and enterprise use, keep temperature low (0.1-0.3). For creative writing, go higher.

Inference

When the model actually runs and produces an answer. Training teaches the model. Inference uses what it learned. When you type a question and the model answers - that's inference. API costs are

inference costs. Latency complaints are inference speed complaints. "Scheduled inference" means running the model automatically at a set time (like Phase 4's daily predictions).

TF-IDF

Term frequency - inverse document frequency. A way to measure which words in a question actually matter. "Show me the budget" - "show" and "me" appear in every question (low importance). "Budget" appears rarely (high importance). TF-IDF gives "budget" a high score and "show" a low score. This is how Phase 4's prediction model knows which words in your past queries are actually predictive.

Feature engineering

Turning raw messy data into clean numbers that a machine learning model can learn from. Your query log has timestamps and text. The ML model can't read those directly. Feature engineering turns "Monday 9:15 AM" into numbers: day_of_week=1, hour=9, is_morning=1. The better your features, the better your model's predictions. This is the real skill in ML - not the algorithm.

The strategic layer - what the bible doesn't cover

Step zero: data readiness

Before you pick any level from the table above, you need to assess: is the data even in a state where AI can use it? This is where 60% of enterprise AI projects die - not in model selection, but in data cleanup. A product leader who walks in and says "before we discuss RAG or fine-tuning, show me where your data lives and in what format" - that person saves the company 6 months of wasted effort.

Questions to ask: Where does the data live? How many systems? What format? Who owns it? Is there PII that needs masking? How current is it? If nobody can answer these in 30 seconds, the data is not ready.

How to measure if AI is working

Level	The question	How to answer it
RAG	Are we retrieving the right documents? Are the answers accurate?	Have 50 employees rate answers weekly. Track: % of answers rated useful.
Fine-tuning	Does the AI sound like us now?	Blind test - employees can't tell if the answer was written by AI or a colleague.
Distillation	Is the small model as good as the big one?	Run the same 1,000 questions through both. Compare answer quality side by side.
ML predictions	Are the predictions actually right?	Of the 10 projects you flagged as risky, how many actually failed? That's precision.
Agent	Does the agent complete the goal correctly end to end?	Give it 20 goals. Track: how many completed fully, how many steps failed, how many results were useful.

Build vs buy vs partner

Approach	When it makes sense	When it doesn't
Build	AI is your core product or a key differentiator. You have engineering talent. You need full control over data, models and iteration speed.	You're building AI because it sounds impressive, not because the problem demands it. You don't have engineers who understand ML.
Buy (SaaS)	Standard use case - customer support, document search, content generation. Someone already built a good product. You want it working this quarter.	Your data is sensitive and can't leave your infrastructure. The vendor's model doesn't understand your domain well enough.

Partner	You need custom AI but don't have the team. A services partner builds it, you own the output. Good for first use case.	You plan to run AI at scale long-term. The knowledge walks out the door when the partner leaves.
----------------	--	--

Who to hire

The most common enterprise mistake: hiring the wrong role for the wrong level.

Role	What they do	Covers	India salary
Software engineer	Builds the RAG pipeline - APIs, data ingestion, vector DB wiring, frontend. Doesn't train models.	Level 1 (RAG) + agent layer	12-35L/year
ML engineer	Runs fine-tuning jobs, manages training pipelines, handles model deployment. Cares about training loss, learning rate, GPU utilization.	Level 2 + 3 (fine-tuning, distillation)	20-50L/year
Data scientist	Builds classical ML models from structured data. Designs features, tests hypotheses, evaluates predictions. Cares about precision, recall and feature importance.	Level 4 (classical ML)	18-45L/year
AI researcher	Designs new architectures, runs pre-training, publishes papers. Don't hire unless you're building a model company.	Level 5 only	40L-1Cr+/year

Common mistake: companies hire data scientists when they need software engineers (for RAG) or hire AI researchers when they need ML engineers (for fine-tuning). Wrong hire = 6 months wasted.

Red flags - what to watch for

These are the signals that an AI project, vendor or internal team is headed for trouble.

"We need our own model."

No you don't. Unless your product IS the model (like Anthropic or Sarvam), you need someone else's model plus your data. RAG + fine-tuning covers 95% of enterprise use cases.

"We'll start with a proof of concept."

POCs without clear success criteria are where budgets go to die. Before starting: define what "working" looks like, who will judge it and what happens if it works. If the answer to the last question is vague, the POC is theater.

"The data is ready."

It almost never is. Ask: where does it live? How many systems? What format? Who owns it? Is there PII that needs masking? How current is it? If nobody can answer these in 30 seconds, the data is not ready.

"We need an AI strategy."

You need a business problem that AI can solve. Start from the problem, not from the technology. "Our customer support team takes 48 hours to resolve tickets - can AI bring that to 4?" is a real question. "We need an AI strategy" is not.

"We need GPUs."

For RAG? No. For fine-tuning with LoRA? Your laptop works. For inference? API calls. GPUs are needed for pre-training models from scratch (level 5) or running very large models locally. Most enterprises are nowhere near needing their own GPU infrastructure.

"Let's hire a Chief AI Officer."

A title won't save you. If you don't have a clear business problem, clean data and engineering capacity, a CAIO will spend 6 months writing PowerPoint decks. Hire for the level you need. Level 1 needs a good software engineer. Not a CxO.

"We need an agent."

Most companies asking for agents haven't finished level 1. An agent orchestrates tools - if the tools don't work well, the agent just orchestrates garbage faster. Get RAG right first. Then fine-tune. Then add the agent layer.

Quick reference - one-liners for each level

Designed to be said out loud, not read from a slide.

RAG: "Don't retrain the model. Just show it the right documents at the right time."

Fine-tuning: "RAG gives it knowledge. Fine-tuning gives it personality."

Distillation: "Make the expensive model teach the cheap model. Then fire the expensive model."

Classical ML: "LLMs are great at language. They're terrible at predicting numbers. Use the right tool."

Agent: "An agent without good tools is a project manager with no team. Fix the tools first."

Own model: "If you're asking whether you need this, you don't."

Part of the Laptop AI project - github.com/swapnil-shah-ai/laptop-ai

Built by learning, not by reading. Every concept on this page was implemented on a regular laptop with no GPU.